

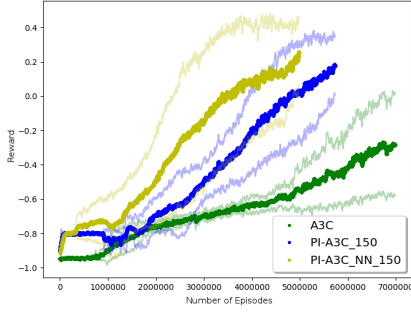


Figure 4: Learning against Rule-based opponent: using the policy head during MCTS rollout phase within the demonstrator provides improvement in learning speed, but it has higher variance compared to the default random policy.

been through different biasing strategies, such as prioritizing actions globally based on their evaluation scores (?), using heuristically computed move urgency values (?), or concurrently learning a rollout policy (?). In a similar vein with these methods, we let the MCTS-based demonstrator to access the policy head during rollouts, named as PI-A3C-NN (Planner Imitation - A3C with Neural Network). Our results suggest that employing a biased rollout policy improves in the average learning performance, however it has higher variance, as depicted in Figure 4.

## Conclusions

In this work, we present a framework that uses a non-expert simulated demonstrator within a distributed asynchronous deep RL method to succeed in hard-exploration domains. Our experiments use the recently proposed Pommerman domain, a very challenging benchmark for pure model-free RL methods. In Pommerman, one main challenge is the high probability of suicide while exploring (yielding a negative reward), which occurs due to the delayed bomb explosion. However, the agent cannot succeed without learning how to stay safe after bomb placement. The methods and ideas proposed in this paper address this hard-exploration challenge. The main idea of our proposed framework is to use MCTS as a shallow demonstrator (small number of rollouts). In our framework, model-free workers learn to safely explore and acquire this skill by imitating a shallow search-based non-expert demonstrator. We performed different experiments varying the quality and the number of demonstrators. The results show that our proposed method shows significant improvement in learning efficiency across different opponents. There are several directions to extend our work. Soemers et al. (?) employed Breadth-First Search to initialize state-action value estimates by identifying near terminal states before starting MCTS based lookahead search. This enhancement can be tested within our framework. Also, MCTS-minimax hybrids method (?) can be employed as the demonstrator, which can provide better tactical play through minimax search. Another avenue is to investigate how to formulate the problem so that *ad-hoc* invocation of an MCTS-based simulator can be employed, i.e., action guidance can

be employed only when the model-free RL agent needs one, e.g., in Pommerman this means whenever a bomb is about to go off or near enemies where enemy engagement requires strategic actions. All of our work presented in the paper is on-policy for better comparison to the standard A3C method — we maintain no experience replay buffer. This means that MCTS actions are used only once to update neural network and thrown away. In contrast, UNREAL uses a buffer and gives higher priority to samples with positive rewards. We could take a similar approach to save demonstrator’s experiences to a buffer and sample based on the rewards.

## Appendix

**Neural Network Architecture:** We use a NN with 4 convolutional layers that have 32 filters and  $3 \times 3$  kernels, with stride and padding of 1, followed with a dense layer with 128 units, followed actor-critic heads. Architecture is not tuned.

**State Representation:** Similar to (?), we maintain 28 feature maps that are constructed from the agent observation. These channels maintain location of walls, wood, power-ups, agents, bombs, and flames. Agents have different properties such as bomb kick, bomb blast radius, and number of bombs. We maintain 3 feature maps for these abilities per agent, in total 12 is used to support up to 4 agents. We also maintain a feature map for the remaining lifetime of flames. All the feature channels can be readily extracted from agent observation except the opponents’ properties and the flames’ remaining lifetime, which are tracked efficiently from sequential observations.

**Hyperparameter Tuning:** We did not perform a thorough hyperparameter tuning due to long training times. We used a  $\gamma = 0.999$  for discount factor. For A3C loss, the default weights are employed, i.e.,  $\lambda_v = 0.5$ ,  $\lambda_\pi = 1.0$ , and  $\lambda_H = 0.01$ . For the *Planner Imitation* task,  $\lambda_{PI} = 1$  is used for the MCTS worker. We employed the Adam optimizer with a learning rate of 0.0001. We found that for the Adam optimizer,  $\epsilon = 1 \times 10^{-5}$  provides a more stable learning curve (less catastrophic forgetting). We used a weight decay of  $1 \times 10^{-5}$  within the Adam optimizer for L2 regularization.

**MCTS Implementation Details:** After rollouts, demonstrator action is selected by the max visit count, a.k.a. *Robust Child* (?). Default UCB1 exploration constant of  $\sqrt{2}$  is used. The max search tree depth from any given state is set to 25 given the low search budget. The value head by the NN is not used by vanilla MCTS.

Velit dolores quod fugit impedit modi iusto expedita, obcaecati pariatur maiores illo dolores sit omnis praesentium officia, assumenda ipsam optio esse nihil quas dicta laboriosam sint explicabo officiis, nam sint tempore fugiat non quaerat quia dolorum atque reiciendis animi, optio numquam earum illum architecto aliquid repellat quam fuga. Voluptas iste id, quia beatae ipsa ut magnam tenetur voluptatem tempora quas corporis vitae, provident eius vero deleniti eum maxime error optio explicabo, voluptates animi obcaecati accusantium saepe. Ad optio eligendi libero ipsum esse nostrum fuga vero temporibus maxime, at nulla odio exercitationem ad beatae. Asperiores nihil odit blanditiis ducimus harum perspiciatis similique veniam laborum, suscipit autem illum tempora laboriosam quae recusandae ullam dolor quo neque earum? Architecto officia facere exercitationem nostrum sequi illo nemo delectus sit, porro voluptate dicta aspernatur, culpa vero amet autem libero rep-

rehenderit, vel iste eius magnam quisquam recusandae perspiciatis  
temporibus quas voluptatem possimus.